Name :- Abhijeet Dinakar Pathade

Roll No :- 16

Division :- TY-A(AIML)

Step No: 1 To Train and Save the Model

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from mpl_toolkits.mplot3d import Axes3D
```

```
df = pd.read_csv("/content/drive/MyDrive/Real estate.csv")
```

```
df.shape
```

```
(414, 8)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 414 entries, 0 to 413
Data columns (total 8 columns):
 #   Column                           Non-Null Count  Dtype  
---  ------                           --------------  -----  
 0   No                               414 non-null    int64  
 1   date                             414 non-null    float64
 2   house age                        414 non-null    float64
 3   distance to the nearest station  414 non-null    float64
 4   number of convenience stores     414 non-null    int64  
 5   latitude                         414 non-null    float64
 6   longitude                        414 non-null    float64
 7   house price of unit area         414 non-null    float64
dtypes: float64(6), int64(2)
memory usage: 26.0 KB
```

```
df.columns
```

```
Index(['No', 'date', 'house age', 'distance to the nearest station',
       'number of convenience stores', 'latitude', 'longitude',
       'house price of unit area'],
      dtype='object')
```

```
#Define features (X) and target (y)
X = df[['house age', 'distance to the nearest station',
       'number of convenience stores', 'latitude', 'longitude']]   # independent variables
y = df['house price of unit area']        # dependent variable
```

```
X.head()
```

	house age	distance to the nearest station	number of convenience stores	latitude	longitude
0	32.0	84.87882	10	24.98298	121.54024
1	19.5	306.59470	9	24.98034	121.53951
2	13.3	561.98450	5	24.98746	121.54391
3	13.3	561.98450	5	24.98746	121.54391
4	5.0	390.56840	5	24.97937	121.54245

```
#Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
#Predictions
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
#Evaluation
print("Coefficients (slopes):", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))
```

```
Coefficients (slopes): [-2.70593236e-01 -4.55249601e-03  1.10512079e+00  2.36092831e+02
 -2.39036942e+01]
Intercept: -2946.6588590244446
Mean Squared Error (MSE): 54.58094520086248
R² Score: 0.6746481382828156
```

```
import pickle
```

```
pickle.dump(model,open('REPPModel.pkl','wb'))
```

Step No: 2 HTML Code for UI

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Real Estate Price Prediction</title>


    <style>

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: Arial, Helvetica, sans-serif;
        }


        body {

            background: #f2f6fc;

            display: flex;

            justify-content: center;

            align-items: center;

            min-height: 100vh;

        }


        .container {

            width: 500px;
```

```
        background: white;

        padding: 35px;

        border-radius: 15px;

        box-shadow:
            0px 8px 25px
            rgba(0, 0, 0, 0.15);

    }


    h1 {

        text-align: center;

        color: #1565c0;

        margin-bottom: 8px;

    }


    h3 {

        text-align: center;

        color: #666;

        margin-bottom: 25px;

        font-weight: normal;

    }


    label {

        display: block;

        font-weight: bold;

        margin-top: 14px;

        margin-bottom: 6px;

        color: #333;

    }


    input {

        width: 100%;

        padding: 11px;

        font-size: 16px;

        border: 1px solid #ccc;

        border-radius: 7px;

        outline: none;

    }


    input:focus {

        border-color: #1565c0;

    }
```

```
        button {
            width: 100%;
            margin-top: 25px;
            padding: 13px;
            background: #1565c0;
            color: white;
            border: none;
            border-radius: 8px;
            font-size: 18px;
            cursor: pointer;
        }

        button:hover {
            background: #0d47a1;
        }

        .result {
            margin-top: 25px;
            padding: 18px;
            background: #e8f5e9;
            color: #2e7d32;
            border-radius: 8px;
            text-align: center;
            font-size: 20px;
            font-weight: bold;
        }

        footer {
            text-align: center;
            margin-top: 20px;
            color: #777;
            font-size: 13px;
        }
    </style>
</head>

<body>

<div class="container">

    <h1>Real Estate Price Predictor</h1>
    <h3>Multiple Linear Regression</h3>
```

```
        <form action="/predict" method="POST">


            <label>
                House Age
            </label>

            <input
                type="number"
                name="house_age"
                step="0.01"
                placeholder="Example: 13.3"
                required
            >


            <label>
                Distance to Nearest Station
            </label>

            <input
                type="number"
                name="distance"
                step="0.01"
                placeholder="Example: 4082.0"
                required
            >


            <label>
                Number of Convenience Stores
            </label>

            <input
                type="number"
                name="convenience_stores"
                min="0"
                step="1"
                placeholder="Example: 2"
                required
            >


            <label>
                Latitude
            </label>

            <input
                type="number"
                name="latitude"
                step="0.000001"
                placeholder="Example: 24.9830"
                required
            >


            <label>
                Longitude
            </label>

            <input
                type="number"
                name="longitude"
                step="0.000001"
                placeholder="Example: 121.5400"
                required
            >


            <button type="submit">

                Predict House Price

            </button>
```

```
    </form>


    {% if prediction_text %}

    <div class="result">

        {{ prediction_text }}

    </div>

    {% endif %}


    <footer>

        By Dr. Sachin Balawant Takmare

    </footer>


</div>


</body>

</html>
```

Step No: 3 Flask Code for Backend

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

# Load the trained model
model = pickle.load(open("REPPModel.pkl", "rb"))


@app.route('/')
def home():
    return render_template("index.html")


@app.route('/predict', methods=['POST'])
def predict():

    # Get input values from HTML form
    house_age = float(request.form['house_age'])

    distance = float(request.form['distance'])

    convenience_stores = float(
        request.form['convenience_stores']
    )

    latitude = float(request.form['latitude'])

    longitude = float(request.form['longitude'])


    # Arrange inputs in EXACTLY the same order
    # used while training the model
    input_data = np.array([[
        house_age,
        distance,
        convenience_stores,
        latitude,
        longitude
    ]])


    # Predict house price
    prediction = model.predict(input_data)
```

```
    # Display prediction
    prediction_text = (
        f"Predicted House Price : "
        f"{prediction[0]:.2f} per unit area"
    )


    return render_template(
        "index.html",
        prediction_text=prediction_text
    )


if __name__ == "__main__":
    app.run(debug=True)
```